

Databases 2 - exam - July 17, 2024 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Concurrency control (12 points, up to 2 points per class)

Consider the schedule $S = r_4(x) \ w_3(x) \ r_4(z) \ w_4(y) \ r_2(x) \ r_1(x) \ w_2(z) \ w_3(y) \ r_2(y) \ w_1(x) \ w_1(y)$. First, classify S with respect to 2PL and TS Multi (with the conventions adopted for TS Multi under Snapshot Isolation, used for the exercises). Then, also classify S with respect to VSR, CSR, Strict 2PL, and TS Mono, but use class inclusion to determine membership wherever possible.

Solution.

Overview

S is 2PL, and thus also CSR, and VSR. S is not Strict 2PL. S is not TS Multi and thus also not TS Mono.

Detailed analysis for the 2PL class

	1	2	3	4	5	6	7	8	9	10	11		
x	$\uparrow_4 r_4$	$\downarrow_4$	$\uparrow_3 w_3$		$\downarrow_3$	$\uparrow_2 r_2$	$\uparrow_1 r_1$		$\downarrow_2$	$\uparrow_1 w_1$	$\downarrow_1$		
y	$\uparrow_4$		w_4	$\downarrow_4$	$\uparrow_3$		w_3	$\downarrow_3$	$\uparrow_2$	r_2	$\downarrow_2$	$\uparrow_1 w_1$	$\downarrow_1$
z	$\uparrow_4$		r_4	$\downarrow_4$		$\uparrow_2 w_2$			$\downarrow_2$				
	4				3				2		1		

Legend

$\uparrow$: read lock request
 $\uparrow$: write lock request
 $\uparrow$: lock upgrade
 $\downarrow$: unlock request

Detailed analysis for the TS Multi class

Operation	RTM(x)	WTM(x)	RTM(y)	WTM(y)	RTM(z)	WTM(z)	Killed transactions
$r_4(x)$	4 (0)	0	0 (0)	0	0 (0)	0	
$w_3(x)$							3 as $3 < \text{RTM}(x) = 4$
$r_4(z)$					4 (0)		
$w_4(y)$				0, 4			
$r_2(x)$	4 (0)						
$r_1(x)$	4 (0)						
$w_2(z)$							2 as $2 < \text{RTM}(z) = 4$
$w_3(y)$							3 (already killed)
$r_2(y)$							2 (already killed)
$w_1(x)$							1 as $1 < \text{RTM}(x) = 4$
$w_1(y)$							1 (already killed)

Detailed analysis for the Strict 2PL class

	1	2	3	4	5	6	7	8	9	10	11
x	r_4	$\downarrow_4$	w_3		$\downarrow_3$	r_2	r_1		w_1		
y			w_4	$\uparrow_4$			w_3	$\uparrow_3$	r_2	w_1	
z			r_4				w_2				

Legend

$\downarrow$: mandatory unlock request
 $\uparrow$: commit time

B. Ranking (9 points, up to 3 points per sub-question)

Consider a dataset with attributes x , y , and z , where high values are better than low values.

$$T_1 =$$

	x	y	z
D :	10	2	4
E :	6	1	5
A :	4	1	2
F :	2	2	3
H :	2	7	4
I :	2	0	5
J :	2	3	1
C :	1	0	5
B :	0	6	0
G :	0	0	4

$$T_2 =$$

	x	y	z
C :	1	0	5
E :	6	1	5
I :	2	0	5
D :	10	2	4
G :	0	0	4
H :	2	7	4
F :	2	2	3
A :	4	1	2
J :	2	3	1
B :	0	6	0

$$T_3 =$$

	x	y	z
D :	10	0	5
E :	5	0	5
A :	5	0	0
H :	0	5	5
J :	0	5	0
B :	0	5	0
F :	0	0	5
I :	0	0	5
C :	0	0	5
G :	0	0	5

1. The table on the left, T_1 , shows the dataset already sorted with respect to x . Use SFS to compute the skyline of T_1 . Make sure to **indicate all dominance tests** made by the algorithm.
2. Consider now table T_2 , shown in the center, which is the same as T_1 but sorted with respect to z . Compute the skyline of T_2 with SFS (no need to show all steps) and comment on the results you obtained.
3. Table T_3 , on the right, has the same data as the other ones, but with values approximated to the closest multiple of 5. Compute its skyline (no need to show all steps) and clarify whether, *in general* (not just with this specific dataset), after the approximation step: *a*) a non-skyline point can become skyline; *b*) a skyline point can become non-skyline.

Solution.

1. The dataset is already sorted, so we do not need to pre-sort it (but see the answer to the following point). There are 16 dominance tests: $D \not\prec E$, $D < A$, $D < F$, $D \not\prec H$, $E \not\prec H$, $D \not\prec I$, $E < I$, $D \not\prec J$, $E \not\prec J$, $H < J$, $D \not\prec C$, $E < C$, $D \not\prec B$, $E \not\prec B$, $H < B$, $D < G$. The skyline is $\{D, E, H\}$.
2. If we proceed as in the previous point, we obtain the skyline $\{C, E, D, H\}$, which unlike the previous one, also contains C . After a closer inspection, we observe that the sorting criterion based solely on z cannot distinguish between C and E (and I) even though C is dominated by E , so placing C before E constitutes a violation of the topological sort principle used by SFS according to which a preceding item cannot be dominated by a subsequent item. We note that the principle is violated in T_1 , too (since F precedes but is dominated by H), but the final result remains correct (F is also dominated by D and thus removed).
3. *a*) In this specific case, the skyline becomes $\{D, H\}$, so a skyline point (like E) can become non-skyline. Indeed, E becomes dominated by D , because its advantage in the z attribute is lost due to the approximation. *b*) No non-skyline point becomes skyline in this example, but this may very well happen: consider a dataset with just two points $\alpha = \langle 2, 2 \rangle$ and $\beta = \langle 1, 1 \rangle$; clearly $\alpha < \beta$, but after the approximation they coincide and thus are both skyline points.

C. Triggers (2+3+3+3 points)

An urban pollution monitoring system monitors the concentration of various pollutants through ground stations. Each ground station monitors the concentration of all the pollutants and inserts in the central database a record only when a change of value is detected. A set of triggers reacts to the insertion of a record by updating the pollution statistics and emitting an alert when threshold values are exceeded at a station. The central database has the following relational schema (primary keys are uppercase):

```
STATION(SID, name, status)
POLLUTANT(PID, name, threshold)
RECORD(SID, PID, TIMESTAMP, value)
STATS(SID, PID, average, peak)
ALERT(SID, TIMESTAMP)
```

Define a set of triggers that react when a ground station inserts a record, as follows:

1. The average value at that station of the pollutant mentioned in the record is updated.
2. If the recorded value exceeds the maximum value sensed in the station for the pollutant, the peak of the pollutant at the station is updated to the recorded value.
3. If the peak value of ANY pollutant exceeds the threshold, the status of the station where the value is sensed is set to *critical*, and an alert tuple is created (or updated, if already existing) to register the last timestamp at which the alert is raised at that station.
4. If 10 subsequent below-threshold values are recorded in a *critical* station after the last alert, the station's status is reset to *normal* and that alert is deleted.

Assumptions:

- Assume that the monitoring is weekly and the data are cleaned up every Monday at 00:00 so that it is not necessary to consider the date and time in the record's timestamp when computing the (weekly) average and peak.
- Timestamps are distinct integer numbers in progressive order of time.
- At the beginning of each week the average and peak values of each pollutant in each station are set to 0 and the status of each station is set to *normal*.

Solution.

```
CREATE TRIGGER T1
AFTER INSERT ON RECORD
```

```
BEGIN
  UPDATE STATS
  SET AVERAGE = (SELECT AVG(value) FROM RECORD
                  WHERE PID = NEW.PID AND SID = NEW.SID)
  WHERE SID = NEW.SID AND PID = NEW.PID;
END;
```

```
CREATE TRIGGER T2
AFTER INSERT ON RECORD
```

```
BEGIN
IF NEW.value >= (SELECT peak FROM STATS WHERE SID = NEW.SID AND PID = NEW.PID) THEN
  UPDATE STATS
  SET peak = NEW.value
  WHERE SID = NEW.SID AND PID = NEW.PID;
END IF;
END;
```

```

CREATE TRIGGER T3
AFTER UPDATE OF peak ON STATS

DECLARE MAX_TS INT;

BEGIN
    SELECT max(timestamp) INTO MAX_TS from record WHERE SID = new.SID;
    IF NEW.peak > (SELECT threshold FROM POLLUTANT WHERE PID = NEW.PID) THEN
        UPDATE STATION
        SET status = 'critical'
        WHERE SID = NEW.SID;

        IF EXISTS (SELECT * FROM ALERT WHERE SID = NEW.SID) THEN
            UPDATE ALERT
            SET timestamp = MAX_TS
            WHERE SID = NEW.SID;
        ELSE
            INSERT INTO ALERT (SID, timestamp)
            VALUES (NEW.SID, MAX_TS);
        END IF;
    END IF;
END;

CREATE TRIGGER T4
AFTER INSERT ON RECORD

WHEN (SELECT STATUS FROM STATION WHERE SID = NEW.SID) = 'critical'

DECLARE
    ALERT_TS INT;

BEGIN
    SELECT timestamp INTO ALERT_TS
    FROM ALERT
    WHERE ALERT.SID = NEW.SID;

    IF (SELECT COUNT(*)
        FROM RECORD
        JOIN POLLUTANT ON RECORD.SID = NEW.SID AND RECORD.PID = POLLUTANT.PID
        WHERE timestamp >= ALERT_TS AND value <= threshold) > 10 THEN
        DELETE FROM ALERT WHERE SID = NEW.SID;

        UPDATE STATION
        SET STATUS = 'normal'
        WHERE SID = NEW.SID;
    END IF;
END;

```